

Arfan Nabil Ramadhan Emerald

THE LAST ASTRONAUT

INPUT n (jumlah astronot)

INPUT k (nilai eliminasi awal)

CREATE array astronot of size n

FOR i FROM 0 TO n-1:

 astronot[i] = i + 1

END FOR

sisa_astronot = n

index_sekarang = 0

PRINT "Urutan eliminasi: "

WHILE sisa_astronot > 1:

 langkah = k - 1

 // Cari astronot ke-K yang masih aktif

 WHILE langkah > 0:

 index_sekarang = (index_sekarang + 1) MOD n

 IF astronot[index_sekarang] != 0:

 langkah = langkah - 1

 END IF

 END WHILE

 tereliminasi = astronot[index_sekarang]

 PRINT tereliminasi

```

// Tandai tereliminasi dengan 0
astronot[index_sekarang] = 0
sis_astronot = sis_astronot - 1

// Perbarui nilai K
IF tereliminasi MOD 2 == 0:
    k = k + 2
ELSE:
    k = k - 1
END IF

IF k < 2:
    k = 2
END IF

// Maju ke astronot berikutnya yang masih hidup
WHILE sis_astronot > 0 AND astronot[index_sekarang] == 0:
    index_sekarang = (index_sekarang + 1) MOD n
END WHILE
END WHILE

// Cari astronot terakhir
FOR i FROM 0 TO n-1:
    IF astronot[i] != 0:
        PRINT "Astronot terakhir: ", astronot[i]
        BREAK
    END IF
END FOR

```

Logika dari masalah ini menggunakan representasi array untuk mencatat status astronot yang masih aktif di dalam lingkaran. Mengingat kita menggunakan pendekatan dasar tanpa struktur data dinamis bawaan, angka 0 digunakan sebagai penanda bahwa astronot di posisi indeks tersebut telah tereliminasi. Perhitungan K-langkah dilakukan dengan iterasi melingkar yang hanya menghitung elemen array yang nilainya bukan 0. Mekanisme ini menyerupai circular list karena indeks array akan berputar kembali ke awal menggunakan operasi modulus ($\text{MOD } n$) ketika mencapai batas elemen terakhir. Setelah seorang astronot tereliminasi, nilai K akan dievaluasi dan diubah secara dinamis. Jika nomor astronot yang gugur bernilai genap, K ditambah 2; jika ganjil, dikurangi 1. Sebuah batas bawah diberlakukan agar K tidak pernah bernilai kurang dari 2. Indeks pencarian kemudian digeser ke posisi astronot aktif tepat setelah posisi eliminasi terakhir. Seluruh perulangan ini terus berjalan hingga variabel penghitung sisa_astronot menunjukkan angka satu, yang merepresentasikan satu-satunya astronot yang tersisa untuk dicetak di akhir program.

ALIEN IN THE MIDDLE

INPUT pesan_asli

panjang_pesan = PANJANG(pesan_asli)

BUAT array hasil_sandi dengan panjang yang sama

IF panjang_pesan > 0 THEN

 // Karakter pertama tidak berubah

 hasil_sandi[0] = pesan_asli[0]

END IF

FOR i = 1 TO panjang_pesan - 1 DO

 // Mengonversi karakter ASCII ke angka 1-26

 nilai_sekarang = ASCII(pesan_asli[i]) - ASCII('A') + 1

 nilai_sebelumnya = ASCII(pesan_asli[i - 1]) - ASCII('A') + 1

 // Menjumlahkan nilai untuk pergeseran sandi

 nilai_baru = nilai_sekarang + nilai_sebelumnya

 // Looping untuk wrap-around jika melebihi batas abjad ('Z')

 WHILE nilai_baru > 26 DO

```
        nilai_baru = nilai_baru - 26
    END WHILE

    // Mengonversi angka kembali ke karakter ASCII kapital
    karakter_baru = CHAR(nilai_baru - 1 + ASCII('A'))
    hasil_sandi[i] = karakter_baru
END FOR

PRINT hasil_sandi
```

Penyelesaian masalah sandi ini berfokus pada manipulasi nilai ASCII karakter huruf kapital. Mengingat adanya pembatasan penggunaan pustaka eksternal pada C++, pesan asli diperlakukan sebagai sebuah deretan (array) karakter. Algoritma melakukan perulangan (looping) dari karakter kedua hingga karakter terakhir pesan. Untuk menghitung nilai pergeseran secara sistematis, setiap huruf terlebih dahulu dikonversi menjadi skala angka dari 1 hingga 26. Proses pemetaan ini dilakukan dengan mengurangi nilai desimal ASCII huruf yang sedang diproses dengan nilai ASCII huruf 'A', kemudian ditambahkan satu. Proses utama dari penyandian (cipherring) adalah menambahkan nilai numerik huruf pada posisi saat ini dengan nilai numerik huruf aslinya pada posisi sebelumnya. Jika hasil penambahan tersebut lebih dari angka 26 (yang berarti melewati huruf 'Z'), sistem akan terus-menerus mengurangi nilai tersebut dengan 26 hingga masuk ke rentang yang valid (1-26). Logika ini secara efektif mensimulasikan perputaran kembali ke huruf 'A'. Terakhir, angka yang sudah sesuai batas abjad tersebut diubah kembali ke bentuk karakter ASCII dan disimpan ke dalam array hasil sebelum akhirnya dicetak.